Proposed Solution Template

Phase 4: Architecture, Schemas & Database Entity-Relationships

PROJECT PLATFORM: MediConnect - Premium MERN Doctor Appointment Platform

REFERENCE PATH: VIP-C2-BOOK-A-DOCTOR

DATE OF EXECUTION: June 16, 2026

AUTHOR / CREATOR: Antigravity AI Engineer & pairing USER

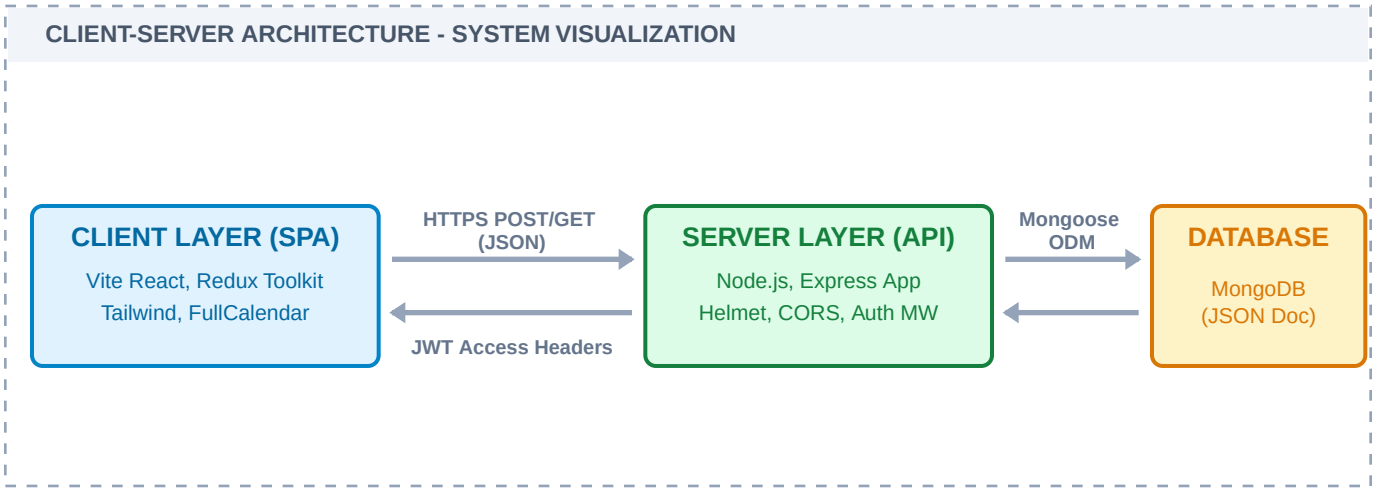
DOCUMENT STATUS: **APPROVED / PRODUCTION-READY**

EXECUTIVE OVERVIEW

This phase-wise document outlines the development lifecycle, system structures, and technical specifications of MediConnect. MediConnect is a clinic-agnostic, hospital-grade scheduling ecosystem built utilizing the MERN stack (MongoDB, Express, React, Node.js). It supports secure JWT access/refresh token structures, comprehensive patient calendars, and simulated multi-modal checkout payment gates.

1. System Architecture

MediConnect is architected as a Client-Server decoupled application leveraging the MERN Stack. The frontend operates as a Single Page Application (SPA) compiled via Vite, using Redux Toolkit for centralized user state and Axios for secure backend communications. The backend is a stateless RESTful API utilizing Express.js running on Node.js, storing state in MongoDB using the Mongoose ODM framework.



2. Database Schema Details

The system utilizes five collections in MongoDB. To ensure referential integrity while retaining NoSQL performance, schemas leverage Mongoose references (`ref`), supplemented by explicit denormalization for fields that are frequently read but rarely updated (like saving doctorSpecialization and doctorName directly inside Appointment documents to speed up patient search layouts).